High Performance Computing

MPI Exercise

Installing MPI

For this exercise we will use an implementation of the **Message Passing Interface (MPI)**.

The recommended implementation is **Open MPI**.

After installation, you should have the following commands available:

```
mpic++
mpirun
mpiexec
```

The command `mpic++` is used to compile C++ programs that use MPI, while `mpirun` or `mpiexec` is used to start several MPI processes.

Linux / Ubuntu

Open a terminal and install the compiler and Open MPI:

```
sudo apt update
sudo apt install build-essential openmpi-bin libopenmpi-dev
```

Check the installation:

```
mpic++ --version
mpirun --version
```

To compile an MPI program:

```
mpic++ -O2 program.cpp -o program
```

To run the program using four MPI processes:

```
mpirun -np 4 ./program
```

You may also use:

```
mpiexec -n 4 ./program
```

Windows — Recommended Method

For Windows users, the recommended approach for this course is to use **Windows Subsystem for Linux (WSL)** with Ubuntu.

Open **PowerShell as Administrator** and run:

```
wsl --install
```

Restart the computer if Windows requests it.

After restarting, open Ubuntu from the Start menu. The first time it starts, Ubuntu will ask you to create a Linux username and password.

Inside the Ubuntu terminal, install Open MPI:

```
sudo apt update
sudo apt install build-essential openmpi-bin libopenmpi-dev
```

Check the installation:

```
mpic++ --version
mpirun --version
```

From this point, Windows students should use the same commands as Linux students.

For example:

```
mpic++ -O2 program.cpp -o program
mpirun -np 4 ./program
```

Important: Run these commands inside the Ubuntu/WSL terminal, not in Windows PowerShell.

macOS

Install Homebrew if it is not already available on your computer.

Then install Open MPI:

```
brew install open-mpi
```

Check the installation:

```
mpic++ --version
mpirun --version
```

Compile and run MPI programs using:

```
mpic++ -O2 program.cpp -o program
mpirun -np 4 ./program
```

Testing the Installation

Create a file called:

```
test_mpi.cpp
```

with the following code:

```cpp
#include <iostream>
#include <mpi.h>

int main(int argc, char* argv[]) {

    MPI_Init(&argc, &argv);

    int rank;
    int size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    std::cout << "Hello from process "
              << rank
              << " of "
              << size
              << std::endl;

    MPI_Finalize();

    return 0;
}
```

Compile:

```
mpic++ -O2 test_mpi.cpp -o test_mpi
```

Run using four processes:

```
mpirun -np 4 ./test_mpi
```

On Windows WSL and Linux, you should see four messages, one from each MPI process. The order of the messages is not guaranteed.

Tasks

The purpose of this exercise is to become familiar with distributed-memory parallel programming using MPI. You will create multiple MPI processes, identify processes using their ranks, perform point-to-point communication, broadcast data from one process to all other processes, combine local results using a reduction operation, and examine how a computational problem can be divided among several processes.

Part 1: MPI Processes and Ranks

Use the program from the installation test.

Run it using:

$$1, \quad 2, \quad 4$$

MPI processes.

Use:

```
mpirun -np 1 ./test_mpi
mpirun -np 2 ./test_mpi
mpirun -np 4 ./test_mpi
```

Complete the table.

Number of Processes	Rank Values
1	
2	
4	

Answer the following questions.

1. What does `MPI_Comm_size()` return?
2. What does `MPI_Comm_rank()` return?
3. If eight MPI processes are started, what are their possible rank values?
4. What is `MPI_COMM_WORLD`?
5. Does every MPI process execute the same program?
6. Does the order in which the processes print their messages always remain the same?

Part 2: Different Work for Different Processes

Create an MPI program where every process calculates

$$x = (\text{rank} + 1)^2.$$

Each process should print its own rank and result.
For example, when four processes are used, the calculations should be:

$$P_0 : 1^2, \qquad P_1 : 2^2, \qquad P_2 : 3^2, \qquad P_3 : 4^2.$$

Start from:

```
#include <iostream>
#include <mpi.h>

int main(int argc, char* argv[]) {

    MPI_Init(&argc, &argv);

    int rank;
    int size;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Calculate a value using the rank


    // Print rank and result


    MPI_Finalize();

    return 0;
}
```

Run the program using four processes.

Questions

1. Does each process have a different value of `rank`?
2. Does every process have its own copy of variable `x`?
3. Does process 0 directly access the variable `x` belonging to process 1?
4. Is any communication required for these calculations?

Part 3: Point-to-Point Communication

MPI processes do not normally access each other's memory directly. Instead, data can be sent explicitly between processes.

For this part, run the program using exactly two processes:

```
mpirun -np 2 ./send_receive
```

Implement the following behavior:

- Process 0 creates an integer with value 100.
- Process 0 sends this integer to process 1.
- Process 1 receives the integer.
- Process 1 multiplies the value by 2.
- Process 1 prints the result.

Use:

```
MPI_Send(...)
```

and

```
MPI_Recv(...)
```

The basic structure is:

```
if (rank == 0) {

    int value = 100;

    // Send value to process 1

}
else if (rank == 1) {

    int value;

    // Receive value from process 0

    value = value * 2;

    std::cout << "Result = "
              << value
              << std::endl;
}
```

For an integer, use the MPI datatype:

`MPI_INT`

You may use message tag:

`0`

Questions

1. Which process is the sender?
2. Which process is the receiver?
3. Why does the receiver need to know the source process?
4. What is the purpose of the message tag?
5. Is this an example of point-to-point or collective communication?

Part 4: Broadcast

Suppose that only process 0 knows the value of N.

Set

$$N = 10,000,000$$

on process 0.

Initially:

```
long long N = 0;

if (rank == 0) {
    N = 10000000;
}
```

Use

```
MPI_Bcast(...)
```

to send the value of N from process 0 to all other processes.

After the broadcast, every process should print:

`Process <rank>: N = 10000000`

Use process 0 as the root process.

Questions

1. Which process initially knows the value of N?
2. Which processes know N after the broadcast?
3. Does `MPI_Bcast()` need to be called only by the root, or by all processes?
4. Why is repeatedly calling `MPI_Send()` from the root less convenient than using a broadcast?

Part 5: Parallel Sum

You will now use several MPI processes to calculate:

$$S = \sum_{i=1}^{N} i.$$

Use

$$N = 10,000,000.$$

Process 0 should define N and broadcast it to all other processes.
Then divide the interval

$$1, \ldots, N$$

among the available MPI processes.
For a process with rank `rank`, you may calculate its range as:

```
long long start =
    rank * N / size + 1;

long long end =
    (rank + 1) * N / size;
```

For example, if

$$N = 100$$

and four processes are used, the approximate decomposition is:

$$P_0 : 1 \ldots 25$$

$$P_1 : 26 \ldots 50$$

$$P_2 : 51 \ldots 75$$

$$P_3 : 76 \ldots 100.$$

Each process should calculate its own local sum:

$$S_p = \sum_{i=\text{start}}^{\text{end}} i.$$

Use a loop to calculate the local sum.
Do **not** use the mathematical summation formula for the local calculation.
Each process should print:

```
Process 0: start = ..., end = ..., local sum = ...
Process 1: start = ..., end = ..., local sum = ...
...
```

Part 6: Reduction

After every process calculates its local sum, combine the results to obtain:

$$S = S_0 + S_1 + \cdots + S_{p-1}.$$

Use:

```
MPI_Reduce(...)
```

with the operation:

`MPI_SUM`

and process 0 as the root.
For the sum variables, use type:

```
long long
```

and MPI datatype:

`MPI_LONG_LONG`

Only process 0 should print the final result.
Verify the result using:

$$S = \frac{N(N+1)}{2}.$$

Questions

1. Why can each process calculate its local sum independently?
2. What information needs to be communicated after the local calculations are finished?
3. What operation does `MPI_Reduce()` perform in this program?
4. Which process receives the final result?
5. What would be different if `MPI_Allreduce()` were used instead?

Part 7: Scatter and Gather

Consider four MPI processes and the following array stored initially on process 0:

$$A = \{10, 20, 30, 40\}.$$

Use:

```
MPI_Scatter(...)
```

so that:

$$P_0 \to 10, \qquad P_1 \to 20, \qquad P_2 \to 30, \qquad P_3 \to 40.$$

Each process should multiply its received value by:

$$\text{rank} + 1.$$

Therefore:

$$P_0 : 10 \times 1,$$

$$P_1 : 20 \times 2,$$

$$P_2 : 30 \times 3,$$

$$P_3 : 40 \times 4.$$

Then use:

```
MPI_Gather(...)
```

to collect the modified values back on process 0.
Process 0 should print the final array.
Run this part using exactly four processes.

Questions

1. What is the main purpose of `MPI_Scatter()`?
2. What is the main purpose of `MPI_Gather()`?
3. Which process contains the complete input array before `MPI_Scatter()`?
4. Which process contains the complete output array after `MPI_Gather()`?

Part 8: Measuring Execution Time

Return to the parallel sum from Parts 5 and 6.
Measure the computation time using:

```
double start = MPI_Wtime();

// parallel computation

double finish = MPI_Wtime();

double elapsed = finish - start;
```

Run the program using:

$$1, \qquad 2, \qquad 4$$

MPI processes.
For this experiment, you may increase N if the execution time is too small to measure reliably.
Record your results.

MPI Processes	Execution Time	Speedup
1		1.00
2		
4		

Calculate speedup as:

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the execution time with one process and T_p is the execution time with p processes.

Questions

1. Does increasing the number of MPI processes always reduce execution time?
2. Is the speedup equal to the number of processes?
3. What additional overhead exists in an MPI program that does not exist in the serial version?
4. Why can communication reduce the benefit of parallel computation?

Part 9: Short Discussion

Answer each question briefly.

1. What is the purpose of `MPI_Init()`?
2. What is the purpose of `MPI_Finalize()`?
3. What is an MPI rank?
4. What is an MPI communicator?
5. What is the difference between point-to-point and collective communication?
6. What is the difference between `MPI_Send()` and `MPI_Bcast()`?
7. What is the difference between `MPI_Reduce()` and `MPI_Gather()`?
8. Why does MPI work well for distributed-memory systems?
9. What is one important difference between MPI processes and OpenMP threads?

Submission

Submit:

1. `hello_mpi.cpp` — MPI process and rank experiment;
2. `send_receive.cpp` — point-to-point communication;
3. `broadcast.cpp` — broadcast experiment;
4. `parallel_sum.cpp` — distributed summation using `MPI_Reduce`;
5. `scatter_gather.cpp` — scatter/gather experiment;
6. the completed execution-time and speedup table;
7. answers to all questions.

All programs must compile using:

```
mpic++ -O2 filename.cpp -o program
```

and must be executable using `mpirun` or `mpiexec`.